

CSCE 641 COMPUTER GRAPHICS · MODULE 12 · LECTURE NOTES

Geometry: Meshes, Curves, and Tessellation

How surfaces are represented, and what each representation makes easy.

MODULE OUTCOMES

- Explain mesh connectivity structures and what queries each makes fast.
- Evaluate Bezier and B-spline curves and state their continuity properties.
- Explain subdivision surfaces and their relationship to splines.
- Describe LOD strategies and the popping problem.
- Explain what the tessellation stages do and when they are worth it.

1 Mesh representations

Triangles are what the rasterizer consumes, but they are a poor structure for almost everything else you might want to do with a surface. The representation you choose determines which operations are cheap.

Structure	Contents	O(1) queries	Use
Triangle soup	9 floats per triangle, no sharing	None	Legacy formats. Avoid: no vertex reuse, no adjacency.
Indexed mesh	Vertex array plus index array	Vertex lookup	The rendering format. Vertices are shared, so the post-transform cache works.
Half-edge	Two directed half-edges per edge, with twin/next/vertex/face links	All adjacency queries	The editing format: simplification, subdivision, smoothing, parameterisation.

1.1 Half-edge

Each undirected edge is represented as two directed half-edges pointing in opposite directions, one belonging to each adjacent face. A half-edge stores its twin, the next half-edge around its face, its origin vertex, and its face. Four pointers, and every adjacency question becomes a constant-time walk:

- **Faces around a vertex:** alternate twin and next until you return to the start.
- **Edges of a face:** follow next until you cycle.
- **The face across an edge:** twin.face.

- **Vertex valence:** count the one-ring walk.

THE MANIFOLD ASSUMPTION

Half-edge assumes the mesh is **manifold**: every edge has exactly two adjacent faces (or one, at a boundary), and the faces around every vertex form a single fan.

Real production assets violate this constantly — T-junctions, duplicated vertices, edges shared by three faces, isolated vertices. Handling or repairing non-manifold input is where most of the engineering in a mesh library actually goes. CSCE 645 treats this properly.

2 Curves

2.1 Bezier

$$\mathbf{B}(t) = \sum_i \mathbf{C}(n, i) t^i (1-t)^{n-i} \mathbf{P}_i$$

A Bézier curve is a weighted average of its control points, with Bernstein polynomials as the weights. Because the weights are non-negative and sum to one at every t , the curve lies entirely within the **convex hull** of its control points — which is what makes conservative bounding and culling of curved geometry possible without evaluating the curve.

The **de Casteljau algorithm** evaluates the curve by repeated linear interpolation: interpolate between adjacent control points at parameter t , then between the results, and so on until one point remains. It is numerically stable, geometrically transparent, and it produces the two control polygons that split the curve at t as a by-product — which is how curves are adaptively subdivided for rendering.

The weakness is global control: every control point's weight is non-zero over the entire parameter range, so moving any one of them alters the whole curve. Raising the degree to add control points makes this worse.

2.2 B-splines and NURBS

B-splines decouple degree from control point count by giving each control point a basis function with **local support** — non-zero only over a few spans. Moving a control point therefore changes only the nearby curve, which is what makes long splines editable. Continuity at the joins between spans is automatic: a degree- n B-spline is C^{n-1} by construction, with no constraints for the user to maintain.

NURBS add a weight per control point, making the curve a *rational* function. The payoff is exact representation of conic sections — circles, ellipses, cylinders — which polynomial splines cannot represent at all. That is why NURBS, not Bézier patches, are the foundation of CAD.

2.3 Continuity

Type	Condition	What it governs
C^0 / G^0	Positions match	No visible gap.

Type	Condition	What it governs
C^1	First derivatives match as functions of t	Smooth <i>and</i> constant speed. Matters for anything travelling along the curve: cameras, animation timing.
G^1	Tangent <i>directions</i> match; magnitudes need not	Looks smooth. This is what the eye judges. A curve can be G^1 without being C^1 , which looks fine but animates with a visible speed jolt.
C^2 / G^2	Curvature matches	Reflections and specular highlights. A G^1 -but-not- G^2 join is invisible in silhouette and shows as a crease in a reflection, which is why car body surfacing demands G^2 .

3 Subdivision surfaces

Rather than defining a surface by a parametric formula, define it as the limit of repeatedly refining a coarse control mesh by a fixed rule. Each step inserts new vertices and repositions existing ones; in the limit the result is a smooth surface, and in practice two or three steps are visually indistinguishable from the limit.

Scheme	Base mesh	Continuity	Used by
Catmull–Clark	Quads (any polygons)	C^2 except at extraordinary vertices, where C^1	Film and games. The dominant scheme.
Loop	Triangles	C^2 except at extraordinary vertices	Triangle-based pipelines.
Doo–Sabin	Any	C^1	Less common.

SUBDIVISION GENERALISES SPLINES

Catmull–Clark applied to a *regular* quad mesh — every vertex of valence 4 — produces exactly a uniform bicubic B-spline surface.

So subdivision did not replace spline patches; it extended them to arbitrary topology. The special behaviour at extraordinary vertices (valence $\neq 4$) is precisely the price of that generality, and it is why modellers are taught to keep quad topology regular where curvature matters.

Practical advantages over patch-based modelling: arbitrary topology with no trimming, a single representation that yields every level of detail, and a control cage that is itself an editable mesh. This is why it became the standard authoring representation for organic models.

4 Level of detail

A model covering nine pixels should not cost fifty thousand triangles. Module 07 gives the sharper version of this: because fragments shade in 2×2 quads, triangles smaller than a few pixels waste most of their shading work on helper lanes. Dense geometry is not merely wasteful, it is *disproportionately* wasteful.

Strategy	Mechanism	Cost
Discrete LOD	Prebuilt variants, switched by screen-space size.	Simple and predictable. Pops visibly at the switch; needs cross-fading or dithering to hide.
Continuous LOD	Progressive mesh: one edge collapse at a time.	No popping, smooth transitions. Complex runtime, awkward on the GPU.
Virtualised geometry	Clusters selected per-cluster per-frame.	Effectively removes popping and LOD authoring entirely. Large runtime system; this is what Nanite-style renderers do.
Impostors	Replace distant geometry with a textured billboard.	Cheapest by far. Breaks under parallax and at close range; good for distant forests and crowds.

4.1 Quadric error simplification

The standard automatic simplification algorithm collapses edges one at a time, cheapest first. The question is how to price a collapse.

Garland and Heckbert's **quadric error metric** assigns each vertex a 4×4 matrix representing the sum of squared distances to the planes of its incident faces. The error of placing a merged vertex at position $\mathbf{v}$ is then $\mathbf{v}^T \mathbf{Q} \mathbf{v}$, and — this is the key property — the quadric of a merged vertex is simply the *sum* of the two original quadrics. Collapse costs are therefore computable in constant time, and the whole simplification runs in $O(n \log n)$ with a priority queue.

The metric preserves silhouettes well, because collapsing an edge on a sharp silhouette incurs large distances to the original planes. It needs explicit extra handling for attribute discontinuities: collapsing across a UV seam or a normal split destroys the texture layout, so those edges must be weighted heavily or forbidden.

5 GPU tessellation

Stage	Invocation	Responsibility
Tessellation control (hull)	Once per patch	Decide the tessellation factors: how finely to subdivide each edge and the interior. This is where adaptive LOD logic lives.
Tessellator	Fixed-function, per patch	Generate a grid of barycentric or UV coordinates at the requested density. Not programmable.
Tessellation evaluation (domain)	Once per generated vertex	Compute the actual position: evaluate the patch, sample a displacement map, apply the surface definition.

Factors can be driven by screen-space edge length, by distance, by local curvature, or by whether an edge lies on the silhouette — silhouette-adaptive tessellation gives most of the visual benefit for a fraction of the triangles, since interior smoothness is already handled by normal mapping.

THE FAILURE MODE

Over-tessellation is one of the most common and most expensive mistakes in real pipelines. Once triangles approach one pixel, quad overshading means up to 75% of fragment-shader work is discarded, and vertex cost has risen to match.

Target triangles of roughly 8–16 pixels. More than that wastes detail; fewer wastes the machine. Measure, do not assume.

Mesh shaders increasingly replace this fixed pipeline with a more flexible compute-like model, but the arithmetic about triangle size is unchanged.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Keenan Crane — Discrete Differential Geometry (free notes, video, code)	The proper treatment of mesh processing and geometry. This is the primary text for CSCE 645; start it here.
GAMES101 Lectures 11–12 — Geometry	Curves, surfaces, subdivision, and simplification at the right level for this module.
libigl tutorial (free)	Working code for half-edge traversal, simplification, smoothing, and parameterisation. Read the source.
Garland & Heckbert — 'Surface Simplification Using Quadric Error Metrics' (free)	The original paper. Short, clear, and still the basis of every production simplifier.
Brian Karis — 'Nanite: A Deep Dive' (SIGGRAPH, free)	How virtualised geometry actually works. Read after the rest of this module.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a half-edge mesh from an indexed triangle mesh. Write and test the one-ring traversal: all faces around a vertex, all neighbours of a vertex, the face across an edge.
- 2 Implement de Casteljau evaluation for a cubic Bézier curve, and use it to subdivide the curve at an arbitrary t . Verify that the two halves reproduce the original.
- 3 Construct two cubic Bézier segments joined G^1 but not C^1 . Animate a point travelling along the pair at constant parameter speed and observe the velocity discontinuity.
- 4 Implement one step of Catmull–Clark subdivision. Apply it to a cube three times and confirm convergence toward a smooth shape. Identify the extraordinary vertices.
- 5 Implement quadric error simplification. Reduce a 50,000-triangle mesh to 5,000 and 500 triangles and capture all three. Note where the silhouette first fails.
- 6 Implement GPU tessellation on a displaced terrain patch with distance-based factors. Measure frame time against tessellation factor and find the point where triangles go sub-pixel and performance collapses.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What adjacency queries does a half-edge structure make $O(1)$, and what assumption does it require that real assets frequently violate?
- 2 Why does a Bézier curve lie within the convex hull of its control points, and what practical use does that property have?
- 3 Give the decisive practical difference between Bézier and B-spline curves, and say why CAD uses NURBS.
- 4 Distinguish C^1 from G^1 continuity, and give a situation where the difference is visible.
- 5 What is the relationship between Catmull–Clark subdivision and bicubic B-spline surfaces?
- 6 Why does the quadric error metric make edge-collapse simplification efficient?
- 7 When does GPU tessellation stop paying for itself, and why specifically?